

Heterogenous Computing Übungsblatt 2

1. Einleitung

Ziel des zweiten Übungsblatts war es, das Verhalten von GPUs zu untersuchen, insbesondere mit Bezug auf das SIMT-Modell und die resultierenden Unterschiede im Vergleich zu CPUs.

In der ersten Aufgabe wurde die Auswirkung von Branch Divergence auf die Rechenleistung untersucht. Die folgende Aufgabe zwei befasst sich mit der, durch die vielen Threads versteckten, Speicherlatenz einer GPU und den verschiedenen Zugriffsmustern.

Das Übungsblatt wurde auf MacOS mit einem M1 AppleSilicon Prozessor bearbeitet. Daraus ergeben sich einige Besonderheiten.

Im Gegensatz zu klassischen GPU-Plattformen, wie zum Beispiel CUDA auf NVIDIA Chips, bietet Apple Silicon ein Unified-Memory-System, bei dem CPU und GPU denselben physischen Speicher teilen. Es wird keine explizite Datenübertragung zwischen Host (CPU) und Device (GPU) nötig. Das vereinfacht die Programmierung, ändert aber auch die Performance Charakteristiken im Vergleich.

Des Weiteren wird für den Kernel die Apple-eigene GPU-API Metal verwendet. Die Metal-Shading-Language ist C++-basiert und ähnlich zu Vulkan und DirectX 12 low-level. Es stehen keine Warp-Level Primitives und nur begrenzt Atomics zur Verfügung, was bei der Implementierung des Kernels entsprechend beachtet werden muss.

Zudem stellt das Metall-Framework weitere Herausforderungen im Bezug auf die Messung. Diese müssen explizit mit `commandBuffer.waitUntilCompleted()` synchronisiert werden.

2. SIMT-Ausführungsmodell und Divergenz

In der ersten Aufgabe des Arbeitsblatts sollten die Unterschiede des SIMT-Modells (Single Instruction, Multiple Threads) mit der CPU nachgewiesen werden.

Dazu benötigt es einen rechenintensiven, aber speicherarmen Kernel, der eine über den Parameter k (Anzahl Interaktionen) konfigurierbare Rechenlast erzeugt. Es sollte dann über eine Spannbreite von Problemgrößen n iteriert werden, um zu messen, ab welchem n ein Warmup-Effekt eintritt und ab wann die optimale Durchsatzrate erreicht wird.

Mittels des Parameters m sollte in einer zweiten Messreihe zudem Branch Divergence simuliert werden, um zu messen, ob und wann es zu nennenswerten Leistungseinbußen durch Warp-Level Divergence kommt. Um einen Vergleich zu schlagen, wurden vergleichbare Messungen auch auf der CPU ausgeführt, die kein SIMT verwendet.

Die Implementierung erfolgte als modulares Swift-Paket mit dem Kernel in Metal. Der Kommandozeilen-Aufruf erlaubt es, die Parameter. Die gemessenen Werte werden mit berechneten Kennzahlen als CSV-Datei geschrieben.

Die Erzeugung der Graphen erfolgte separat auf Grundlage dieser Daten mit Python.

Die Parameter sind:

- n , die Anzahl an zu berechnenden Elementen, jeweils ein Thread
- k , die arithmetische Last pro Thread in Form der Anzahl an Schleifeniterationen
- tpg , die Menge an Threads pro Gruppe
- m , der Divergenzgrad in Zweierpotenzen

Um die Messungen vor Rauschen durch anderweitige Systemlast zu schützen und eine Verzerrung der Ergebnisse durch die Frequenzregulation zu vermeiden, die bei Apple Silicon besonders aggressiv sind, werden für jeden Datenpunkt der Messreihe jeweils 3 Warmup-Iterationen ausgeführt, denen 10 Messiterationen folgen. Als Ergebniswert wird das arithmetische Mittel der 10 Messungen angenommen.

Zudem muss eine ausreichende Menge an Last erzeugt werden. Für k wurde deshalb ein Wert von 2048 Interaktionen pro Thread angenommen. Der optimale Wert für tpg wurde in einer ersten Kalibrierungsmessreihe ermittelt.

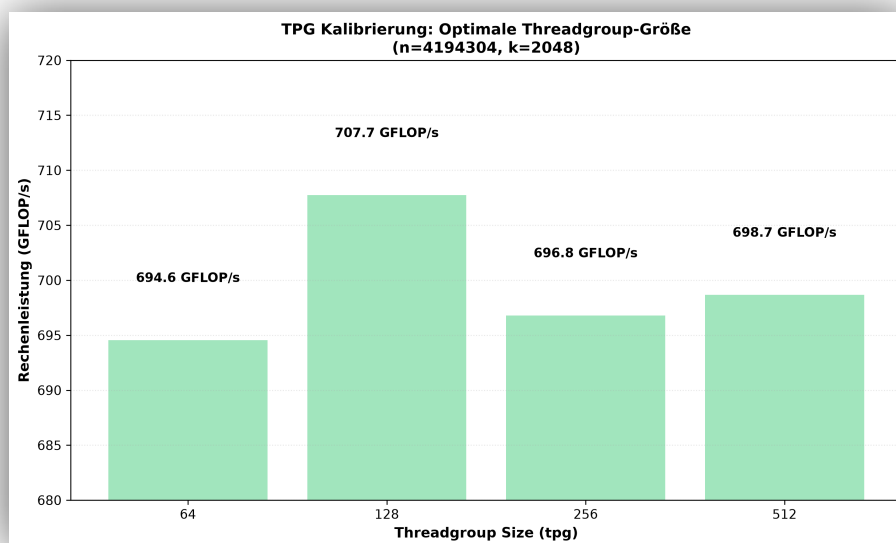


Abb. 1 TPG Kalibrierung

Obwohl die Unterschiede im Bereich von ca. 1,9% liegen und damit klein sind, wurde die höchste Rechenleistung mit einer Threadgroup-Größe von 128 Threads erreicht. Diese wird in den folgenden Messungen ebenfalls verwendet.

Der Grund für die bessere Performance bei $tpg = 128$ liegt vermutlich in einer optimalen Balance zwischen Ressourcenauslastung und Scheduling-Overhead auf der M1 GPU-Architektur. Bei zu kleinen Threadgroup-Größen kann nicht die volle Parallelisierungsfähigkeit genutzt werden, zu große Threadgroups hingegen können andere Faktoren wie Cache-Effektivität oder Speicher-Bandbreite limitieren.

Etwas anders ist das Format der CPU-Messungen. Da das Benchmarking auf dieser deutlich länger dauert, dafür weniger Warmup benötigt, wird nur ein Lauf zum Aufwärmen angenommen und verworfen. Der Datenpunkt errechnet sich aus dem Durchschnitt von 3 Messungen.

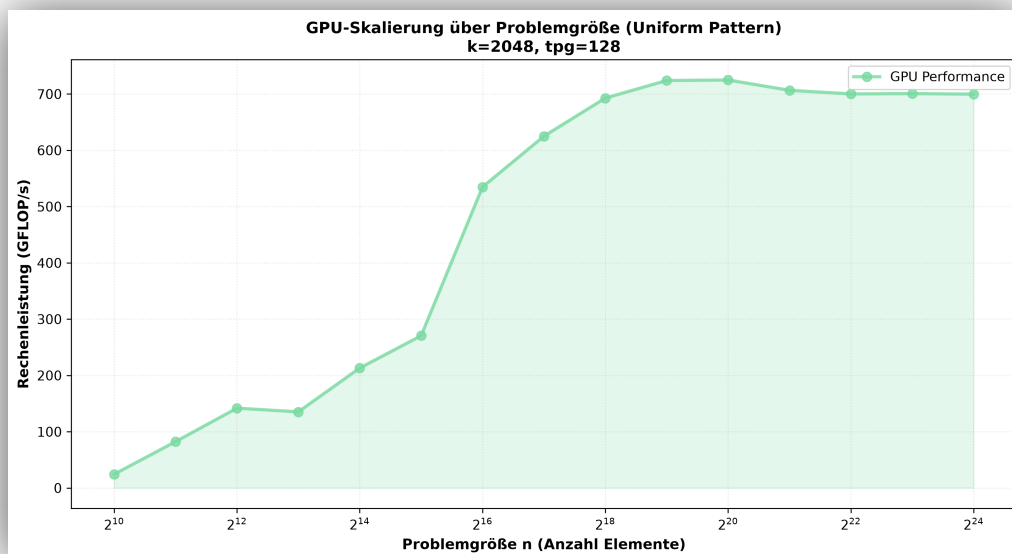


Abb. 2 GPU-Skalierung über Problemgröße (Uniform Kernel)

Als erste Messungen wurde ein Sweep auf der GPU über steigende n, von n^{10} bis n^{24} ausgeführt, noch ohne Divergenz.

Die Messergebnisse zeigen den erwarteten Effekt deutlich. Bei kleinen Problemgrößen, also wenigen Threads, ist die Rechenleistung sehr niedrig. Für $n=1024$ Elemente wurden lediglich 24.505 GFLOP/s gemessen. Mit zunehmender Problemgröße steigt die Performance an, bei $n=2^{18}=262.144$ Elementen sind bereits 95% der theoretischen Maximalleistung erreicht, bei $n=2^{20}=1.048.576$ wird mit 724.914 GFLOP/s der Maximalwert erreicht, bevor ein Plateau Effekt eintritt.

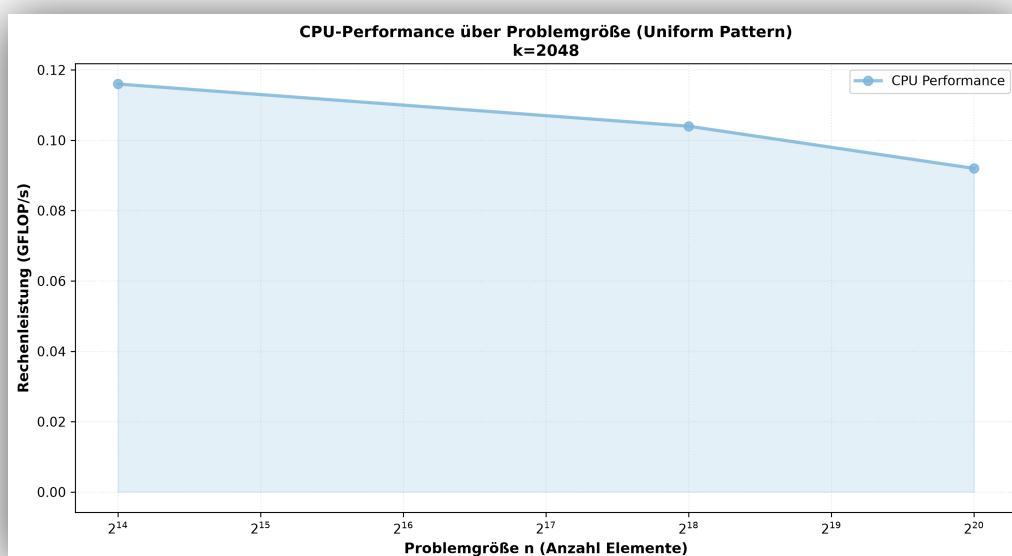


Abb. 3 CPU-Performance über Problemgröße (Uniform Kernel)

Solche Effekte sind bei der CPU, wie erwartet, nicht zu verzeichnen. Die Leistung fällt ab, je mehr Elemente bearbeitet werden müssen.

Ein zweites Experiment sollte die Divergenz veranschaulichen. Mit Festem $n=2^{22}=4.194.304$ und steigenden Divergenzgraden $m \in \{1, 2, 4, 8, 16, 32\}$ sind deutliche Performance-Einbrüche zu erwarten.

Ein erster Messlauf bestätigte diese Erwartung jedoch nicht. Nach Analyse des Kernel-Codes stellte sich heraus, dass der Compiler aggressive If-Conversion und Predication-Optimierungen vornahm, die den Kontrollfluss in bedingte Instruktionen umwandelte. Dadurch wurden beide Pfade spekulativ ausgeführt, sodass keine echte Divergenz auftrat.

Um das Problem zu beheben, wurde der verschaltete if-else-Block durch einen Switch ersetzt und die Inline-Funktionen ausgelagert sowie mit der einer Compilerflag `[[clang::noinline]]` versehen.

Abb. 3 Die Compileroptimierung wird umgangen

Anschließend wurden brauchbare Messergebnisse erzeugt, wenngleich der Wert für $m=1$ aufgrund der wegfallenden Optimierung bedeutend schlechter ausfiel als unter Verwendung des uniformen Kernels.

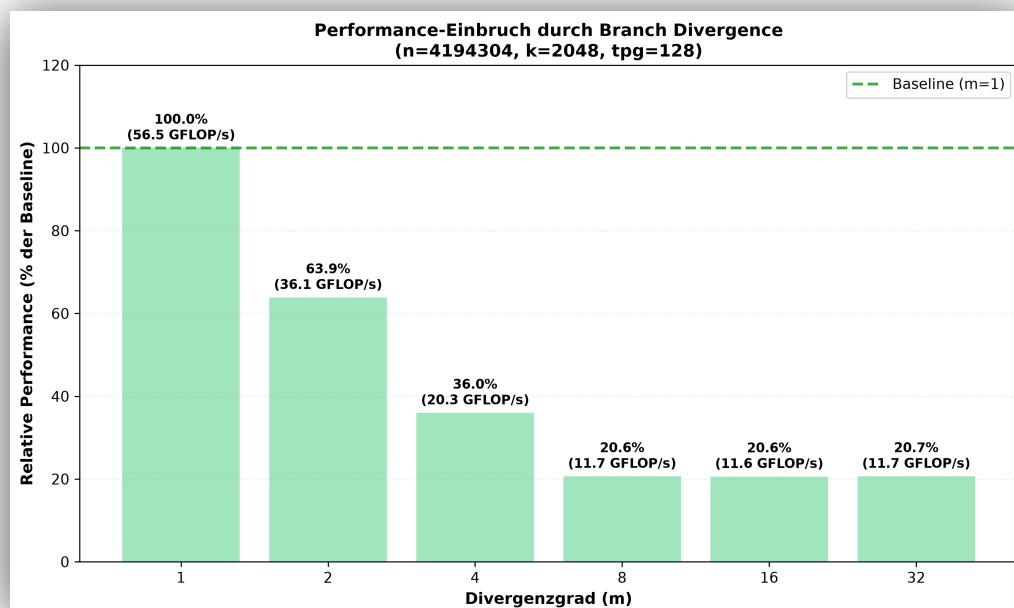


Abb. 4 Branch Divergence auf der GPU (Divergent Kernel)

Nach dieser Korrektur spiegeln die Ergebnisse das erwartete Bild wieder. Für $m=1$ wurde eine Rechenleistung von 56.5 GFLOP/s verzeichnet. Die fällt bei $m=2$ bereits auf ca 64% der Baseline, bei $m=8$ werden gerade noch 20.6% erreicht. Interessanterweise zeigen sich bei höheren Divergenzgraden keine weiteren Verschlechterungen mehr.

Der vergleichende CPU-Benchmark zeigt erneut eine deutlich niedrigere Performance. Mit angepassten Parametern wird aber deutlich, dass kein Performance-Einbruch durch Branch Divergence zu beobachten ist.

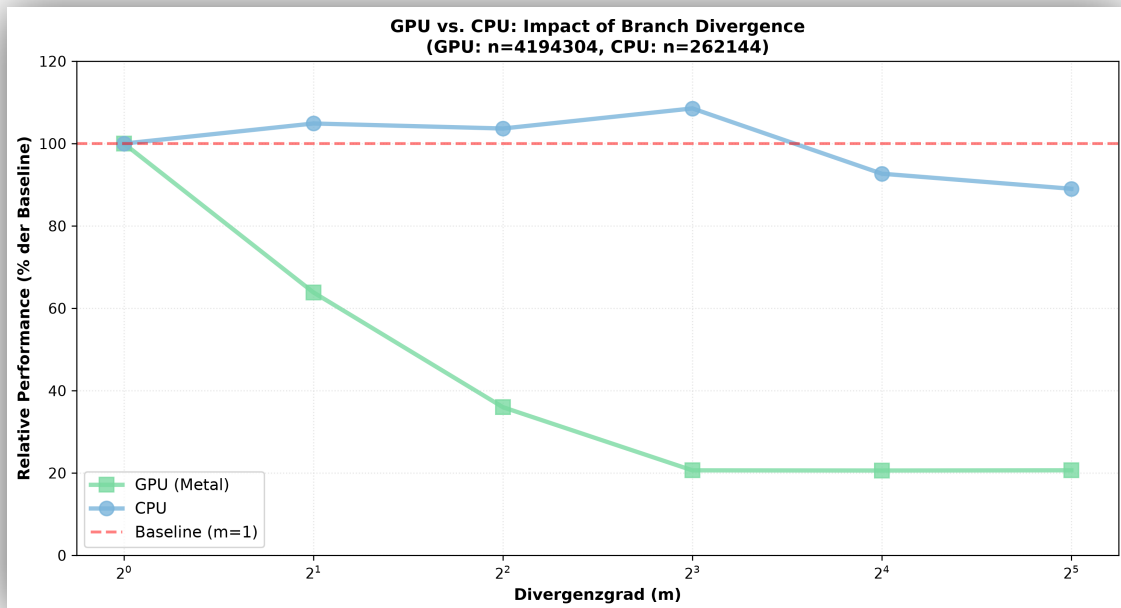


Abb. 5 Vergleich von GPU und CPU anhand der jeweiligen Baseline (Divergent Kernel)

3. Speicherlatenz und Bandbreite

Die zweite Aufgabe sollte zeigen, dass die GPU Latenz beim Speicherzugriff durch hohe Parallelität verstecken kann. Es wurde ein speicherintensiver Kernel mit geringer Rechenlast implementiert, um drei Zugriffsmuster zu vergleichen:

- Coalesced, Threads lesen nebeneinander liegende Speicheradressen
- Stride k, Threads lesen Speicheradressen die k weit auseinander liegen
- gather, Threads lesen Speicheradressen nach einer zufälligen Festlegung

Auch hierbei ergab sich eine Besonderheit durch den Metal Kernel auf Apple M1. Da es keine Entsprechung der CUDA-Occupancy in Metal gibt, wird Die GPU-Auslastung stattdessen durch eine Occupancy-Proxy-Metrik gemessen, die das Verhältnis der aktiven SIMD-Gruppen zur maximalen Anzahl möglicher Groups pro Threadgroup angibt. Die Metrik wird aus den Metal-Runtime-Eigenschaften abgeleitet und liefert eine verwertbare Näherung auf Threadgroup-Ebene.

Ebenso gibt es unter MacOS kein klassisches DeviceQuery zur Abfrage der maximalen Speicherbandbreite. Diese ist stattdessen den Apple-SoC-Spezifikationen zu entnehmen und beträgt für den verwendeten M1-Chip 68.25 GB/s.

Gemessen wurde mit 3 Warmups und 20 Wiederholungen.

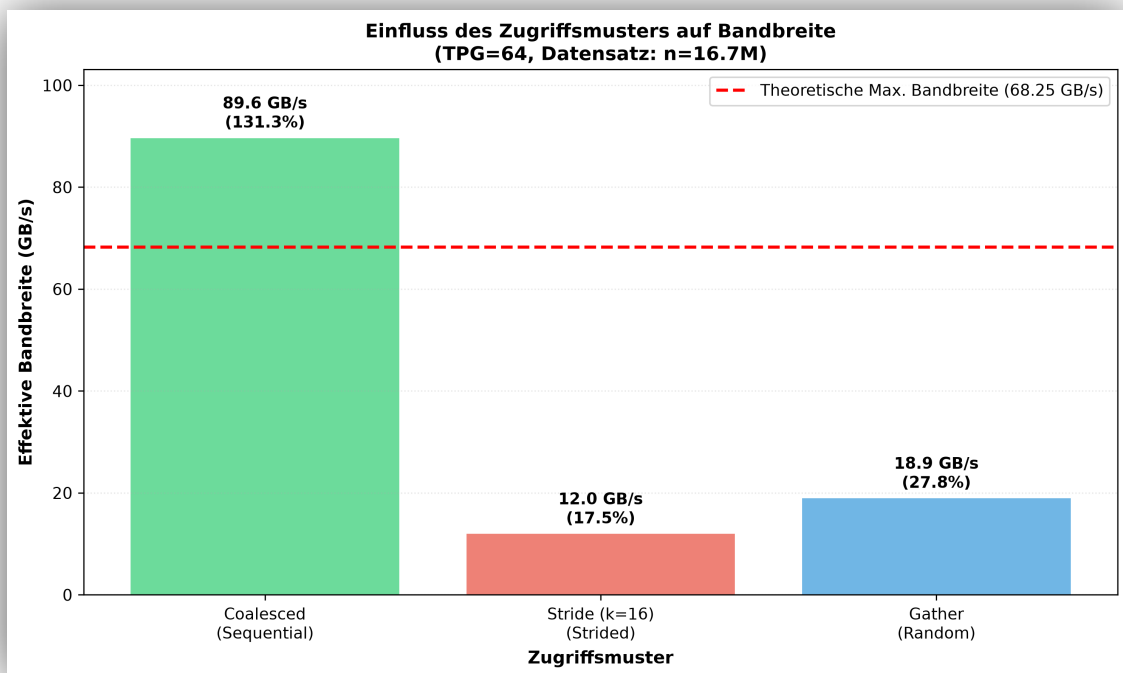


Abb. 5 Die drei Zugriffsmuster im Vergleich

Es ist zu beobachten, dass das Zugriffsmuster die Speicherbandbreite klar beeinflusst. Mit der coalesced Strategie wächst die gemessene Bandbreite sogar über das theoretische Maximum hinaus. Die anderen beiden Strategien fallen deutlich langsamer aus. Gather erreicht nur ca. 28% des theoretischen Maximums. Die Strategie stride mit $k=16$ ist nochmal langsamer und kommt auf gerade einmal 17,5% der theoretischen Leistung.

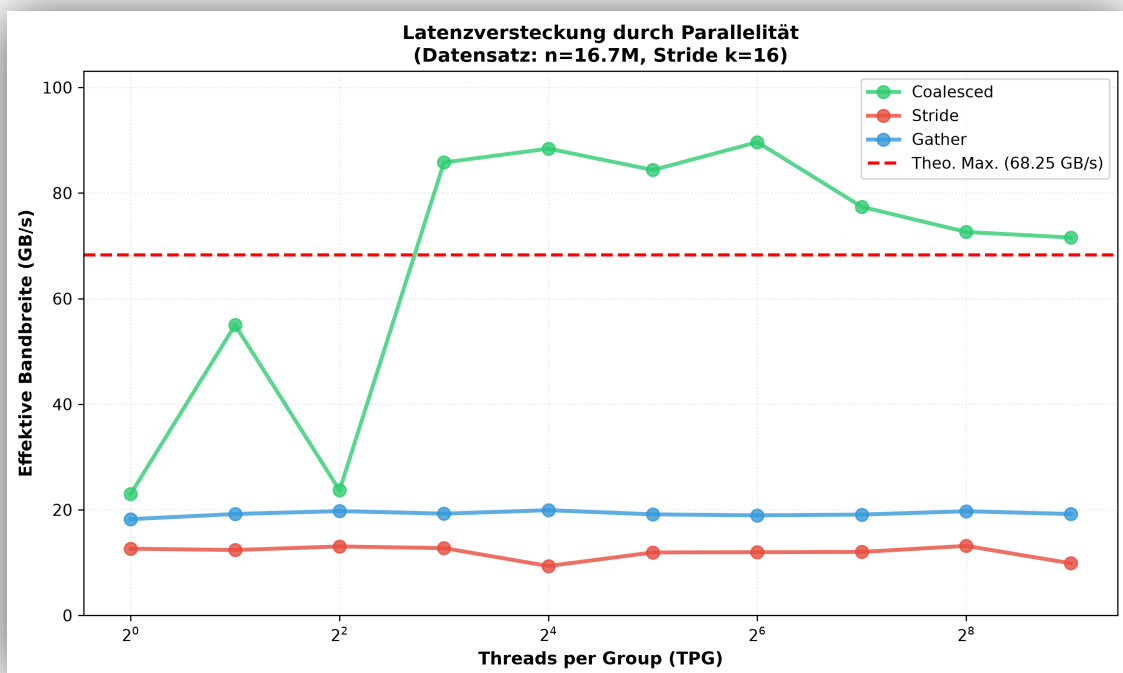


Abb. 5 Verhalten der Zugriffsmuster bei verschiedener tpg

Bei der Messung mit verschiedener Threadgroup-Größen zeigt sich bei Coalesced tatsächlich ein starker Anstieg von 2^0 bis 2^3 , die Ergebnisse sind danach etwa gleich, bis der Datendurchsatz ab 2^6 wieder sinkt. Bei den anderen beiden Mustern ist keine nennenswerte Veränderung zu erkennen.

Die GPU profitiert also stark von regelmäßigen, zusammenhängenden Speicherzugriffen und der großen Anzahl parallel agierender Threads, sodass die tatsächliche Latenz der Speicherzugriffe dem Nutzer verborgen bleibt.

Eine CPU erreicht Ähnliches auf anderem Weg. Über Caches, Prefetching und Out-of-Order-Ausführung wird die Latenz auch hier verborgen, allerdings kann es durch Cache-Misses bei zufälligem Zugriff zu kurzzeitigen Einbrüchen in der Bandbreite und damit wahrnehmbarer Verzögerung kommen.

Aus den Messergebnissen wird deutlich, dass es für den Datenzugriff auf der GPU relevant ist, von den natürlichen Vorteilen Gebrauch zu machen. Daten sollten so angeordnet werden, dass benachbarte Threads auf benachbarte Adressen im Speicher schreiben. Randomisierte Zugriffe sollten vermieden werden.